Development Document

ES — Software Engineering

March 2, 2026

Available Frameworks

Three mobile development frameworks were suggested for this class. The pros, cons, and thought process behind the final choice of technology will be discussed briefly in the following sections.

React Native

React Native is an open-source UI software framework developed by Meta (formerly known as Facebook) used primarily to develop applications for Android and iOS, with additional community-supported extensions for other platforms. React Native uses JavaScript and the React library to build cross-platform applications using a single codebase. It aims to address user experience problems associated with web-view-based frameworks by enabling communication between JavaScript logic and native rendering modules through an abstraction layer, with ongoing architectural improvements aimed at reducing performance overhead.

Pros

- **Cross-Platform Development**
- **Ecosystem**
  React Native has a considerable community and is one of the most active open-source communities on GitHub, with countless open-source packages created and maintained every day.
- **Native UI Components**
  Uses platform-native components for a more familiar user experience.

Cons

- **Performance Overhead**
  Communication between JavaScript and native modules can impact performance due to bridge overhead.
- **Platform Inconsistencies**
  UI behavior may vary slightly between platforms as a result of minor differences in native components.
- **Reliance on Community-Maintained Packages**
  Advanced features often rely on community-maintained packages, which may hinder long-term stability.

Flutter

Flutter is an open-source framework used for developing modern user interfaces for applications that run on multiple platforms such as mobile, web, and desktop. It is based on the Dart programming language and uses its own rendering engine (Skia) to ensure consistent visual output across platforms.

Pros

- **Beginner Friendly**
  Deemed online as a good starting mobile framework for beginners.
- **Quick Development**
  One codebase can be used for multiple platforms.
- **High Performance**
  Ahead-of-time compilation and hardware-accelerated rendering via Skia contribute to smooth animations and competitive runtime performance.
- **Cohesive Appearance and Granular Control**
  By rendering its own UI, Flutter ensures consistent, deterministic design and deep customization.
- **Hot Reload**
  Code changes can be reflected instantly during development while preserving app state.

Cons

- **Size**
  Since apps are shipped with their own rendering engine, app bundle size tends to be larger.
- **Non-Native UX**
  Since it renders its own UI components rather than relying directly on platform-native elements, platform-specific design nuances must be implemented explicitly.
- **Small developer community**
  Dart has lower adoption compared to other programming languages used for the same purpose, such as JavaScript in other frameworks. Which limits the job marked and third-party ecosystem.

Kotlin Multiplatform

Kotlin Multiplatform is a cross-platform development technology created by JetBrains based on the Kotlin programming language. It allows developers to share business logic across platforms such as Android, iOS, and desktop. It primarily focuses on sharing business logic rather than providing a unified cross-platform UI toolkit. Compilation depends on the target platform, including JVM bytecode generation for Android and JVM environments, native binary compilation for supported platforms, and JavaScript output for web targets.

Pros

- **Native UI Flexibility**
  Allows developers to use native UI frameworks for each platform.
- **Strong Performance**
  Compiles to platform-optimized binaries on supported platforms.

- **Incremental Adoption in Existing Projects**
  An application can gradually be rewritten in Kotlin without the need to switch the entire codebase at once.

Cons

- **Smaller Community Compared to Flutter or React Native**
- **UI Not Included**
  Kotlin Multiplatform does not provide built-in UI components. Developers must build separate interfaces for each supported platform which makes it a pretty much disqualifying for a team with no experience.
- **Steeper Learning Curve**
  Requires understanding of shared module architecture and platform interoperability.

Final Choice

The group has limited experience with mobile development and limited familiarity with all the programming languages used by the evaluated frameworks. Considering this constraint, as well as the availability of team members, a rapid project kick-off with sufficient room for experimentation is crucial.

This means choosing a framework with good documentation and a considerable amount of learning resources to quickly produce prototypes, experiments and tests. Options like Flutter and React Native are appealing because of features such as Hot Reload and Fast Refresh, as well as the large amount of available educational content online.

Additionally, the hundreds of widgets that ship with Flutter provide significant benefits for the rapid development of functional prototypes and reduce dependencies on third-party packages compared to React Native.

Given the context and objectives of the project, Flutter aligns closely with the team's current skill level, time constraints, and requirement for rapid prototyping. Although Dart is not as widely adopted and the team lacks prior knowledge of the language, the Flutter's structure, documentation quality, and comprehensive tooling help mitigate this risk.